DECK 3 OF 3

SECURITY MONITORING

Wazuh | Snort | Zeek. Detect. Analyze. Respond.

Wazuh (host) | Snort (network IDS/IPS) | Zeek (network recorder)

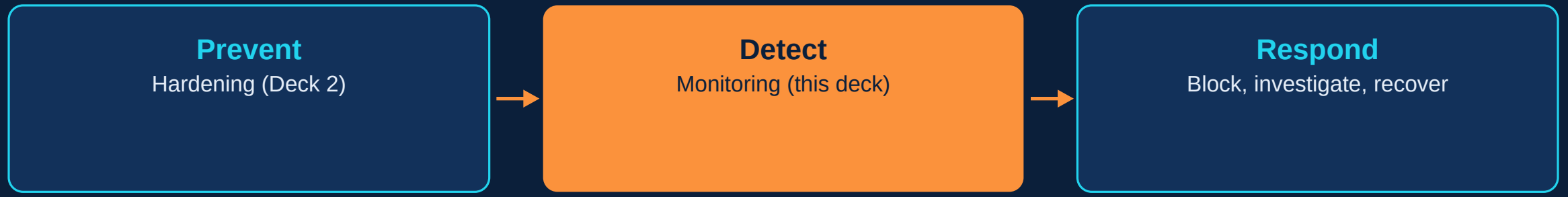
Presented by: Tyrone Justine A. Medina

OVERVIEW

Agenda

- **Part 1: Why monitoring**
  Prevent, detect, respond. SIEM, IDS, IPS and NSM explained
- **Part 2: Wazuh**
  Host-based SIEM and XDR: install, agents, FIM, rules, active response
- **Part 3: Snort**
  Signature-based network IDS/IPS: modes, config, rules, commands
- **Part 4: Zeek**
  Network security monitor: logs, deployment, analysis, scripts
- **Part 5: Putting it together**
  Comparison, layered setup, alert triage workflow

Why Monitoring?



- **Assume breach**
  no prevention is perfect
- **Dwell time**
  the sooner you detect, the smaller the damage
- Without logs and network records you cannot answer "what happened?"

Four Terms You Must Know

SIEM

Collects and analyzes logs from many sources, correlates events and raises alerts.

IDS

Intrusion Detection System. Detects and alerts, does not block.

IPS

Intrusion Prevention System. Detects and blocks, sits inline.

NSM

Network Security Monitoring. Records network activity for investigation.

> *Host-based tools watch what happens inside a server (Wazuh). Network-based tools watch the traffic between devices (Snort, Zeek).*

What Do We Actually Watch?

Source	What it tells you	Tool
/var/log/auth.log	Logins, sudo, SSH failures	Wazuh, Fail2ban
Files in /etc and /usr/bin	Tampering, persistence	Wazuh FIM
Network packets	Scans, exploits, known malware traffic	Snort
Connection metadata	Who talked to whom, DNS, TLS	Zeek
Installed package versions	Known vulnerabilities (CVE)	Wazuh vulnerability detection
Configuration state	Drift from a CIS baseline	Wazuh SCA

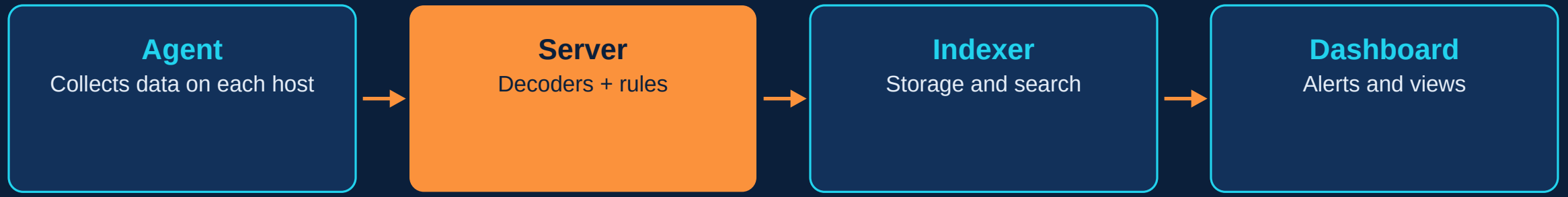
Wazuh: Overview

- Open-source security platform: SIEM + XDR
- Host-based: lightweight agent on each endpoint
- Supports Linux, Windows, macOS, cloud and containers
- Forked from OSSEC, maps detections to MITRE ATT&CK
- Free, with a large community and thousands of built-in rules

Quick facts

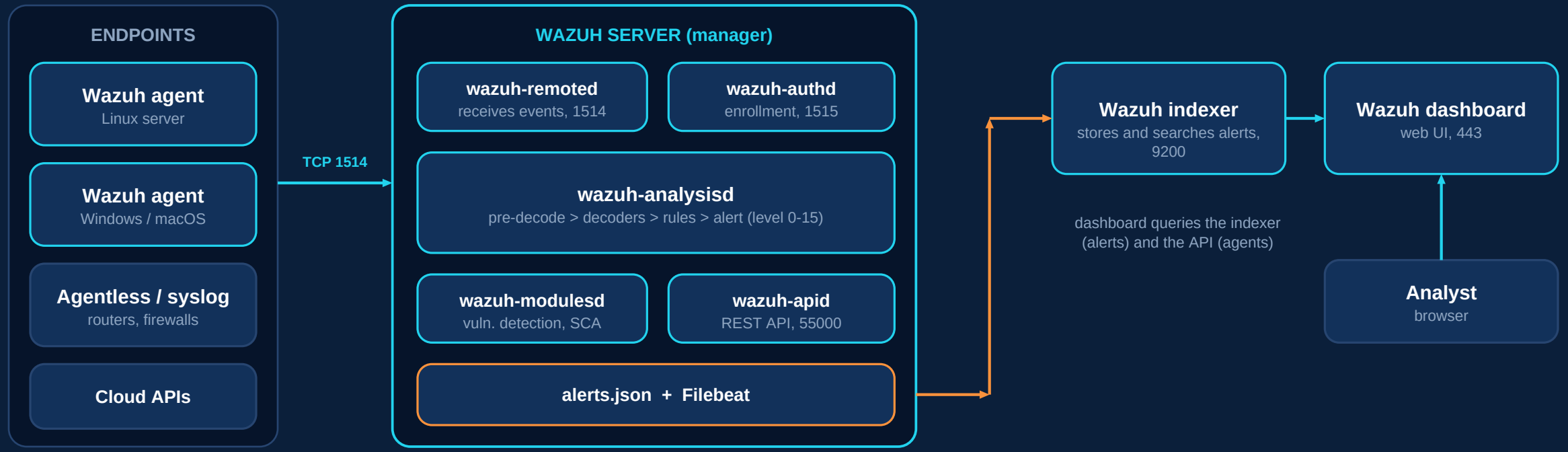
Agents: on every endpoint
Rules: thousands built in
Platforms: Linux / Win / Mac / Cloud
License: Open Source

Wazuh: Architecture



- Agents send events to the server over an encrypted channel
- The server decodes each log line, then matches it against rules
- The indexer stores alerts, the dashboard is where analysts work
- A small lab can run all four components on one machine (all-in-one)

Wazuh: Architecture in Detail



Agents only collect

Decoding and rule matching happen on the server, so rules are updated in one place and agents stay light.

Keys and groups

Each agent gets a unique key at enrollment (1515). Group agents (web, db) to push shared settings from the manager.

Scale out

Lab: all-in-one on one VM. Production: server cluster and multi-node indexer behind a load balancer.

Wazuh: What the Agent Collects

Module	What it does	Where it is configured	Example finding
Log collection	Reads log files, Windows events and command output and forwards them	`<localfile>`	Failed SSH login in auth.log
FIM (syscheck)	Hashes files and watches paths, permissions and owners	`<syscheck>`	sshd_config modified
Rootcheck	Looks for known rootkit traces and hidden processes or ports	`<rootcheck>`	Hidden process found
SCA	Scans settings against CIS-style policies	`<sca>`	PermitRootLogin should be no
Inventory (syscollector)	Lists packages, processes, ports, OS and hardware	`<wodle name="syscollector">`	openssl 3.0.2 installed
Active response	Runs a script when the manager says so	`<active-response> (manager)`	IP blocked for 10 min

agent: /var/ossec/etc/ossec.conf

```
# Add one more log file to watch
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/nginx/error.log</location>
</localfile>
# sudo systemctl restart wazuh-agent
```

Settings can also be pushed centrally: put them in /var/ossec/etc/shared/<group>/agent.conf on the manager and every agent in that group receives them.

Wazuh: Capabilities

Log Analysis

Rules and decoders turn raw log lines into meaningful alerts.

File Integrity

Detects changes in /etc, binaries and web roots.

Vulnerabilities

Matches installed packages against known CVEs.

SCA

Security Configuration Assessment against CIS benchmarks.

Active Response

Automatically blocks an IP or runs a script.

Compliance

Maps alerts to PCI DSS, GDPR, HIPAA and more.

Wazuh: Ports to Know

Port	Protocol	Used for
1514	TCP	Agents sending events to the server
1515	TCP	Agent enrollment (registration)
55000	TCP	Wazuh server API
9200	TCP	Wazuh indexer API
443	TCP	Dashboard (HTTPS)

> *Open 1514 and 1515 only from your agent networks, and keep 9200 and 55000 internal.*

Wazuh: Server Installation

INSTALL

```
# Server: all-in-one install
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash ./wazuh-install.sh -a

# Read the generated admin password
sudo tar -O -xvf wazuh-install-files.tar \
  wazuh-install-files/wazuh-passwords.txt

# Check the services
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-indexer wazuh-dashboard

# Dashboard: https://<server-ip>   user: admin
```

- **Lab sizing**
  roughly 4 vCPU and 8 GB RAM for a small all-in-one
- **Version**
  the 4.14 in the URL changes, check the Wazuh docs for the latest
- Change default passwords and keep the install files safe

Wazuh Install, Step 1: Run the Installer

tj@wazuh-server: ~

```
$ cat /etc/os-release | grep PRETTY_NAME
PRETTY_NAME="Ubuntu 22.04.5 LTS"
$ nproc && free -h | grep Mem
4
Mem:           7.8Gi       412Mi       6.9Gi       1.0Mi       612Mi       7.2Gi
$ curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
$ sudo bash ./wazuh-install.sh -a
```

wazuh-install.sh -a (output shortened)

```
07/11/2026 10:02:11 INFO: Starting Wazuh installation assistant. Wazuh version: 4.14.x
07/11/2026 10:02:11 INFO: Verbose logging redirected to /var/log/wazuh-install.log
07/11/2026 10:02:19 INFO: --- Dependencies ---
07/11/2026 10:02:41 INFO: --- Wazuh indexer ---
07/11/2026 10:02:41 INFO: Starting Wazuh indexer installation.
07/11/2026 10:04:12 INFO: Wazuh indexer installation finished.
07/11/2026 10:04:40 INFO: Wazuh indexer cluster initialized.
07/11/2026 10:04:41 INFO: --- Wazuh server ---
07/11/2026 10:04:41 INFO: Starting the Wazuh manager installation.
07/11/2026 10:05:58 INFO: wazuh-manager service started.
07/11/2026 10:06:03 INFO: Filebeat installation finished.
07/11/2026 10:06:20 INFO: --- Wazuh dashboard ---
07/11/2026 10:07:31 INFO: wazuh-dashboard service started.
07/11/2026 10:07:49 INFO: Wazuh dashboard web application initialized.
07/11/2026 10:07:49 INFO: --- Summary ---
07/11/2026 10:07:49 INFO: You can access the web interface https://<wazuh-dashboard-ip>:443
    User: admin
    Password: <ADMIN_PASSWORD>
07/11/2026 10:07:49 INFO: Installation finished.
```

Illustrative screenshot: sample output, values and versions will differ on your system.

1. **Check the basics**
   Supported 64-bit Linux, about 4 vCPU and 8 GB RAM for a small lab, and a static IP.
2. **Download the assistant**
   One script installs all components. The 4.14 in the URL is the release, check the docs for the latest.
3. **-a means all-in-one**
   Indexer, server, Filebeat and dashboard go on one machine. Use -i to ignore minimum-hardware checks in tiny labs only.
4. **Wait for the Summary**
   It prints the admin user and a generated password once. Copy it now.

> *The whole install takes roughly 5 to 15 minutes depending on your CPU and internet speed.*

Wazuh Install, Step 2: Credentials and Checks

tj@wazuh-server: ~

```
$ sudo tar -O -xvf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt
# Admin user for the web user interface and Wazuh indexer.
  indexer_username: 'admin'
  indexer_password: '**********************'
# Wazuh API
  api_username: 'wazuh'
  api_password: '**********************'
  api_username: 'wazuh-wui'
```

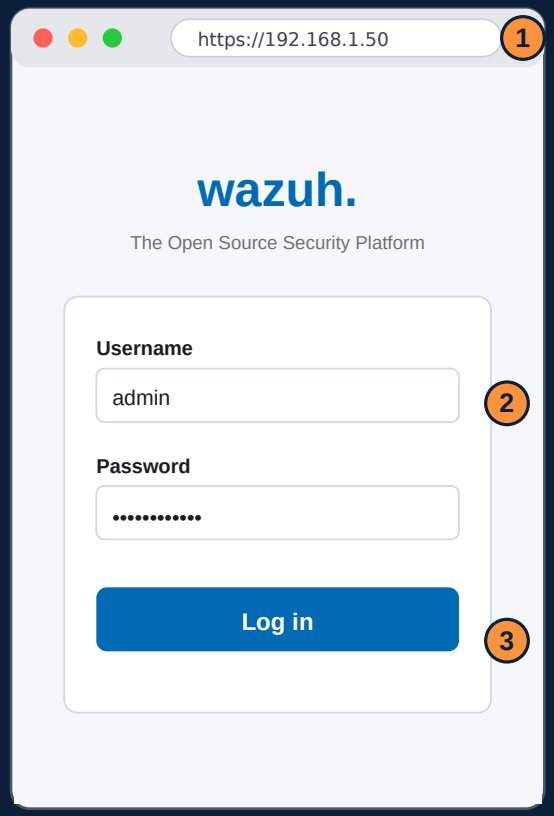
tj@wazuh-server: ~

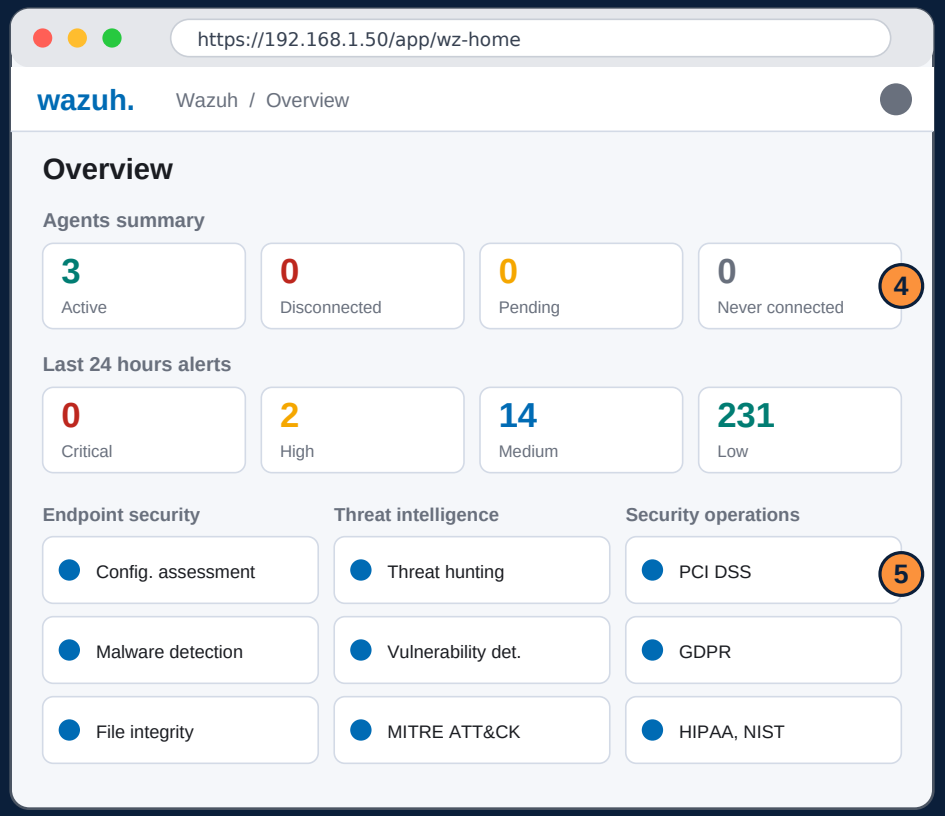
```
$ sudo systemctl status wazuh-indexer wazuh-manager wazuh-dashboard filebeat | grep -E "^●|Active"
● wazuh-indexer.service - wazuh-indexer
     Active: active (running) since Sat 2026-11-07 10:04:12 UTC; 9min ago
● wazuh-manager.service - wazuh-manager
     Active: active (running) since Sat 2026-11-07 10:05:58 UTC; 8min ago
● wazuh-dashboard.service - wazuh-dashboard
     Active: active (running) since Sat 2026-11-07 10:07:31 UTC; 6min ago
● filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch.
     Active: active (running) since Sat 2026-11-07 10:06:03 UTC; 8min ago
$ sudo ss -tlnp | grep -E ':(1514|1515|55000|9200|443) '
LISTEN 0  128  0.0.0.0:1514   0.0.0.0:*  users:(("wazuh-remoted",pid=7211,fd=5))
LISTEN 0  128  0.0.0.0:1515   0.0.0.0:*  users:(("wazuh-authd",pid=7160,fd=4))
LISTEN 0  128  0.0.0.0:55000  0.0.0.0:*  users:(("python3",pid=7342,fd=7))
LISTEN 0  128  *:9200         *:*        users:(("java",pid=3880,fd=325))
LISTEN 0  511  0.0.0.0:443    0.0.0.0:*  users:(("node",pid=8120,fd=20))
```

Illustrative screenshot: sample output, values and versions will differ on your system.

1. **Save the passwords**
   They live inside wazuh-install-files.tar. Store it somewhere safe and do not leave it on a shared folder.
2. **Rotate defaults**
   Wazuh ships a passwords tool (wazuh-passwords-tool.sh) to change admin and API passwords, see the docs.
3. **All four services running**
   Indexer, manager, dashboard and Filebeat must all say active (running).
4. **Ports are listening**
   Each port in the ports slide should show a process. A missing 9200 or 55000 points to that component.

Wazuh Install, Step 3: First Login





1. **Open the dashboard**
   Browse to https://<server-ip>. Port 443 must be reachable from your PC.
2. **User: admin**
   Use the indexer admin password from the Summary or wazuh-passwords.txt.
3. **Log in**
   You land on the Overview page. It is empty until agents report in.
4. **Status cards**
   Agents by state, and the last 24 hours of alerts by severity (critical is level 15 and up).
5. **Module groups**
   Endpoint security, threat intelligence and compliance modules. FIM, SCA and vulnerability detection live here.

First visit: "Your connection is not private"

The installer creates a self-signed certificate, so the browser warns you. In a lab choose Advanced > Proceed. For real use, install a certificate from your own CA.

Illustrative screenshot: sample output, values and versions will differ on your system.

Wazuh: Agents and Manager Commands

ENDPOINT (AGENT)

```
# After adding the Wazuh repo
sudo WAZUH_MANAGER="10.0.0.5" \
  apt install wazuh-agent
sudo systemctl daemon-reload
sudo systemctl enable --now wazuh-agent

# Agent log
sudo tail -f /var/ossec/logs/ossec.log
```

MANAGER

```
# List connected agents
sudo /var/ossec/bin/agent_control -l

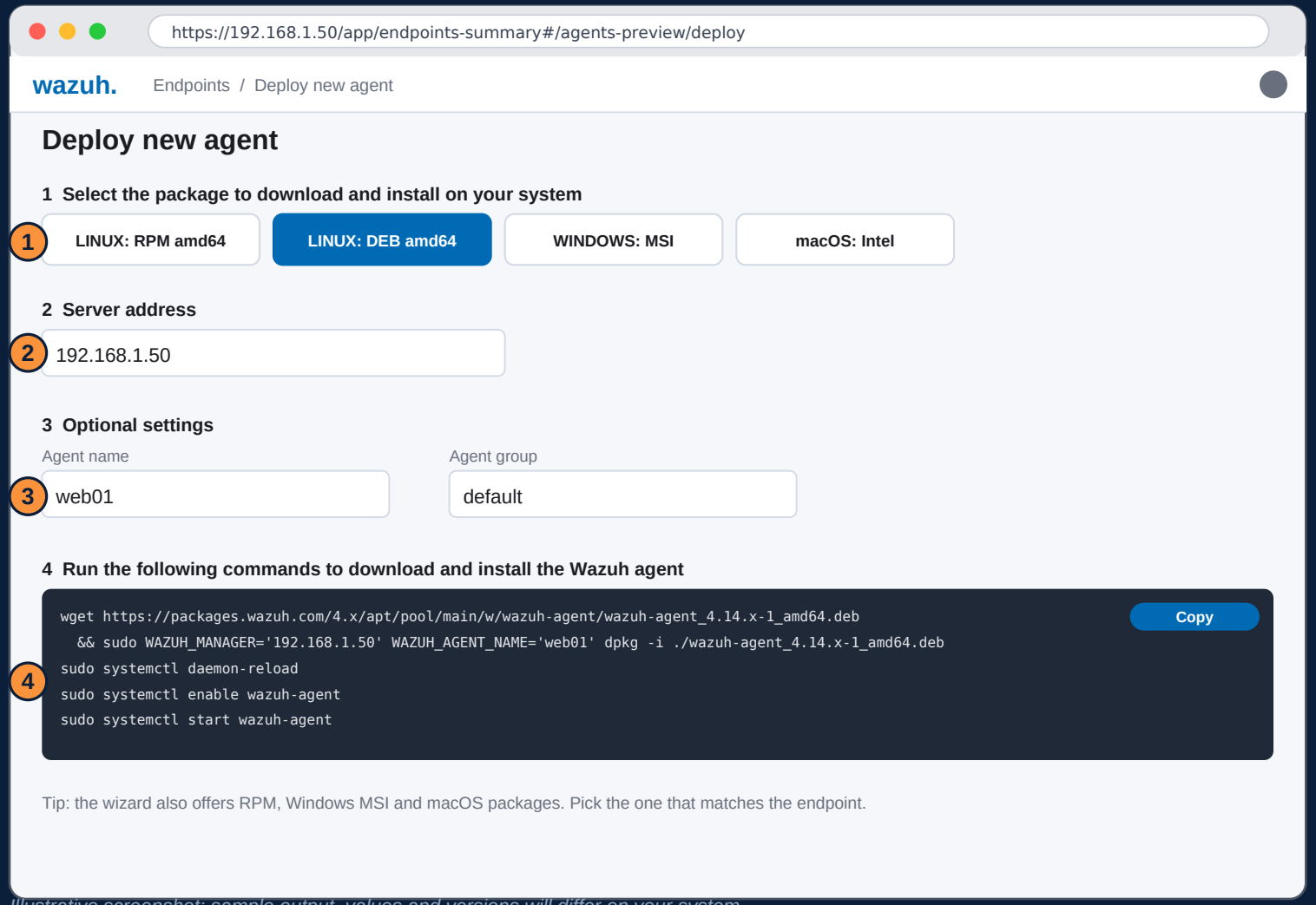
# Test how a log line is decoded
sudo /var/ossec/bin/wazuh-logtest

# Watch alerts live
sudo tail -f /var/ossec/logs/alerts/alerts.json

# Service status
sudo /var/ossec/bin/wazuh-control status
```

> *Use wazuh-logtest every time you write or change a rule. Paste a real log line and see which rule fires.*

Enrolling an Agent: Dashboard Wizard



Illustrative screenshot: sample output, values and versions will differ on your system.

1. **Pick the OS and package**
   The wizard builds the right command for DEB, RPM, Windows or macOS.
2. **Server address**
   IP or DNS name of the manager. Agents must reach it on 1514 and 1515.
3. **Name and group**
   Use clear names (web01, db01). Groups decide which shared config an agent gets.
4. **Run it on the endpoint**
   Paste the commands as root on the server you want to monitor. It installs, enrolls and starts the agent.

> *The original deck uses apt install after adding the repo. The wizard uses dpkg, both end with the same agent.*

Verify the Agent is Connected

web01 (agent)

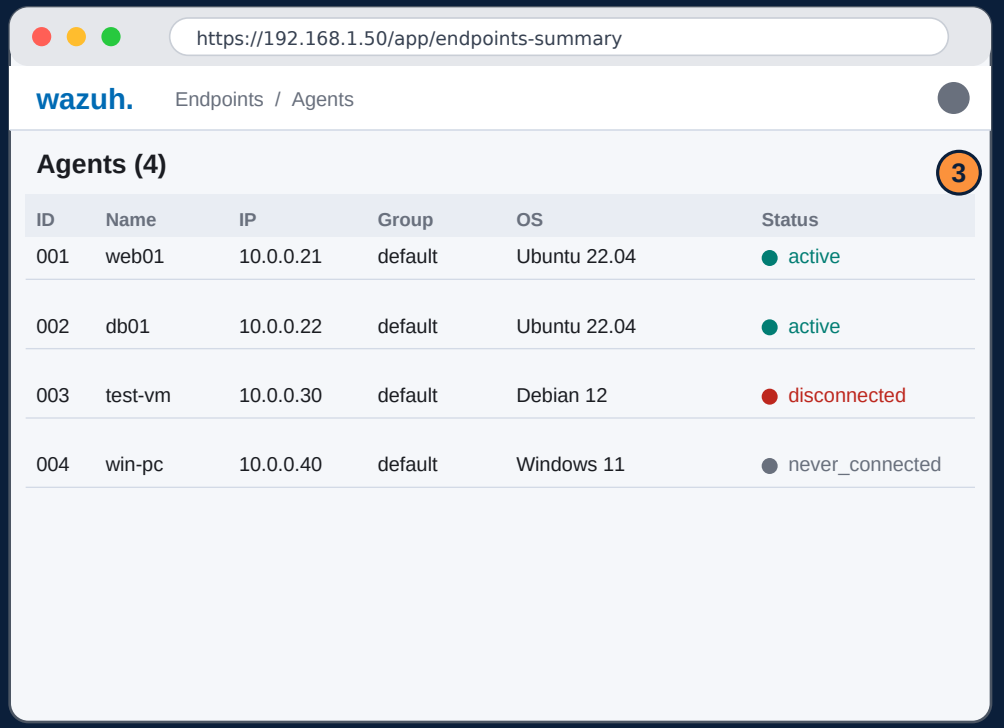
```
$ sudo tail -n 6 /var/ossec/logs/ossec.log
2026/11/07 10:21:05 wazuh-agentd: INFO: Requesting a key from server: 192.168.1.50
2026/11/07 10:21:05 wazuh-agentd: INFO: Using agent name as: web01
2026/11/07 10:21:05 wazuh-agentd: INFO: Waiting for server reply
2026/11/07 10:21:06 wazuh-agentd: INFO: Valid key received
2026/11/07 10:21:08 wazuh-agentd: INFO: (4102): Connected to the server
   ([192.168.1.50]:1514/tcp).
```

1

wazuh-server (manager)

```
$ sudo /var/ossec/bin/agent_control -l
Wazuh agent_control. List of available agents:
   ID: 000, Name: wazuh-server (server), IP: 127.0.0.1, Active/Local
   ID: 001, Name: web01, IP: any, Active
   ID: 002, Name: db01, IP: any, Active
   ID: 003, Name: test-vm, IP: any, Disconnected
```

2



Agent shows "Never connected" or "Disconnected"? Check in this order

1 Network
nc -zv <manager> 1514 and nc -zv <manager> 1515

2 Config
WAZUH_MANAGER value, or <address> in agent ossec.conf

3 Service
systemctl status wazuh-agent, then read ossec.log

4 Clock and version
Keep time in sync. Agent version must not be newer than the manager.

Illustrative screenshot: sample output, values and versions will differ on your system.

1 agent log 2 manager list 3 dashboard status

Wazuh: File Integrity Monitoring

FIM

```
<!-- ossec.conf on the agent -->
<syscheck>
  <frequency>43200</frequency>
  <directories check_all="yes" realtime="yes">
    /etc,/usr/bin
  </directories>
  <directories check_all="yes" report_changes="yes">
    /var/www
  </directories>
  <ignore>/etc/mtab</ignore>
</syscheck>

sudo systemctl restart wazuh-agent
```

- **realtime**
  alert immediately instead of waiting for the next scan
- **report_changes**
  shows what changed inside text files
- **Use case**
  an alert if sshd_config changes after a brute-force attempt
- Monitor important paths only, noisy paths create alert fatigue

FIM in Action: Seeing a Change

web01 (agent): simulate a suspicious edit

```
# Lab only: append a line to a monitored web file (/var/www has report_changes)
$ echo '<?php system($_GET["c"]); ?>' | sudo tee -a /var/www/html/index.php
<?php system($_GET["c"]); ?>
```

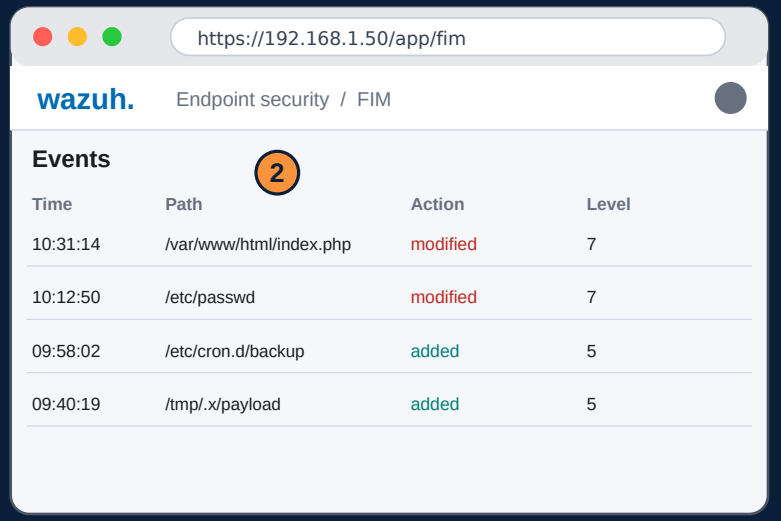
1

wazuh-server: last alert (trimmed with jq)

```
$ sudo tail -n 1 /var/ossec/logs/alerts/alerts.json | jq '{rule, agent: .agent.name, syscheck}'
{
  "rule": { "id": "550", "level": 7, "description": "Integrity checksum changed.",
    "groups": ["ossec", "syscheck", "syscheck_entry_modified", "syscheck_file"] },
  "agent": "web01",
  "syscheck": {
    "path": "/var/www/html/index.php",
    "event": "modified", "mode": "scheduled",
    "changed_attributes": ["size", "mtime", "md5", "sha1", "sha256"],
    "size_before": "412", "size_after": "446",
    "md5_before": "9c1b...e07a",  "md5_after": "41d8...b3f2",
    "diff": "5a6\n> <?php system($_GET[\"c\"]); ?>"
  }
}
```

3

This is the classic web shell pattern: a one-line PHP file change on a web root. Because /var/www uses report_changes, the alert shows exactly what was added.



1 **The change**
A real edit on a watched path. FIM sees it either at once (realtime) or at the next scan (frequency).

2 **The alert**
Rule 550 means checksum changed. New files use 554, deleted files use 553.

3 **The evidence**
Old and new hashes, size and the text diff.

Illustrative screenshot: sample output, values and versions will differ on your system.

Wazuh: A Custom Rule

CUSTOM RULE

```
<!-- /var/ossec/etc/rules/local_rules.xml -->
<group name="local,syslog,">
  <rule id="100100" level="10">
    <if_sid>5402</if_sid>
    <match>/etc/shadow</match>
    <description>Sudo used on /etc/shadow</description>
  </rule>
</group>

# Test with a sample log line, then apply
sudo /var/ossec/bin/wazuh-logtest
sudo systemctl restart wazuh-manager
```

- **Rule IDs**
  use 100000 and above for your own rules
- **if_sid**
  builds on another rule, here 5402 (successful sudo to root)
- **level**
  0 to 15, higher means more severe
- **Decoders**
  extract fields from raw logs before rules run

Rules and Decoders: Test with wazuh-logtest

wazuh-server: sudo /var/ossec/bin/wazuh-logtest

```
Type one log per line
Nov  7 10:15:22 web01 sshd[2411]: Failed password for root from 203.0.113.50 port 51122 ssh2

**Phase 1: Completed pre-decoding.
    timestamp: 'Nov  7 10:15:22'
    hostname: 'web01'
    program_name: 'sshd'
**Phase 2: Completed decoding.
    name: 'sshd'
    dstuser: 'root'
    srcip: '203.0.113.50'
    srcport: '51122'
**Phase 3: Completed filtering (rules).
    id: '5760'      level: '5'
    description: 'sshd: authentication failed.'
    groups: '['syslog', 'sshd', 'authentication_failed']'
**Alert to be generated.
```

same tool, now testing the custom rule 100100

```
Nov  7 10:30:11 web01 sudo:   tj : TTY=pts/0 ; PWD=/home/tj ; USER=root ; COMMAND=/usr/bin/cat /etc/shadow
**Phase 2: Completed decoding.
    name: 'sudo'
    dstuser: 'root'  srcuser: 'tj'
**Phase 3: Completed filtering (rules).
    id: '100100'    level: '10'
    description: 'Sudo used on /etc/shadow'
**Alert to be generated.
```

Illustrative screenshot: sample output, values and versions will differ on your system.

1. **Phase 1: pre-decoding**
   Splits the standard syslog header into time, host and program.
2. **Phase 2: decoding**
   A decoder pulls out fields such as srcip and dstuser. Rules can then test these fields.
3. **Phase 3: rules**
   Wazuh finds the best matching rule and prints its ID, level and groups. No match means no alert.

- **Where files live**
  Custom rules: /var/ossec/etc/rules/local_rules.xml. Custom decoders: /var/ossec/etc/decoders/local_decoder.xml. Never edit the built-in files, updates overwrite them.

> *Workflow: paste real log, adjust rule, test again, then restart the manager.*

Wazuh: Active Response

ACTIVE RESPONSE

```
<!-- manager ossec.conf: auto-block SSH brute force -->
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5712</rules_id>
  <timeout>600</timeout>
</active-response>

sudo systemctl restart wazuh-manager
```

- **firewall-drop**
  blocks the source IP with the host firewall
- **location local**
  runs on the agent that saw the attack
- **timeout 600**
  removes the block after 10 minutes
- Whitelist your admin IPs so you never block yourself

Active Response in Action



web01 (agent): sudo tail -n 4 /var/ossec/logs/active-responses.log

```
$ sudo tail -n 4 /var/ossec/logs/active-responses.log
2026/11/07 10:41:07 active-response/bin/firewall-drop: Starting
2026/11/07 10:41:07 active-response/bin/firewall-drop: "command":"add","srcip":"10.0.0.77"
2026/11/07 10:51:07 active-response/bin/firewall-drop: Starting
2026/11/07 10:51:07 active-response/bin/firewall-drop: "command":"delete","srcip":"10.0.0.77"
```

web01 (agent): the block is just a firewall rule

```
$ sudo iptables -L INPUT -n --line-numbers
Chain INPUT (policy ACCEPT)
num  target  prot opt source        destination
1    DROP    all  --  10.0.0.77     0.0.0.0/0
# ... 600 seconds later the line is gone
$ sudo iptables -L INPUT -n --line-numbers | grep -c 10.0.0.77
0
```

Log format is simplified here and differs between Wazuh versions. Test only in your own lab.

1. **Trigger**
   Active response only reacts to the rule IDs you list. Start with high-confidence rules.
2. **Command**
   firewall-drop edits the host firewall on the agent that saw the attack (location local).
3. **Timeout**
   The block removes itself, so one mistake does not lock someone out forever.

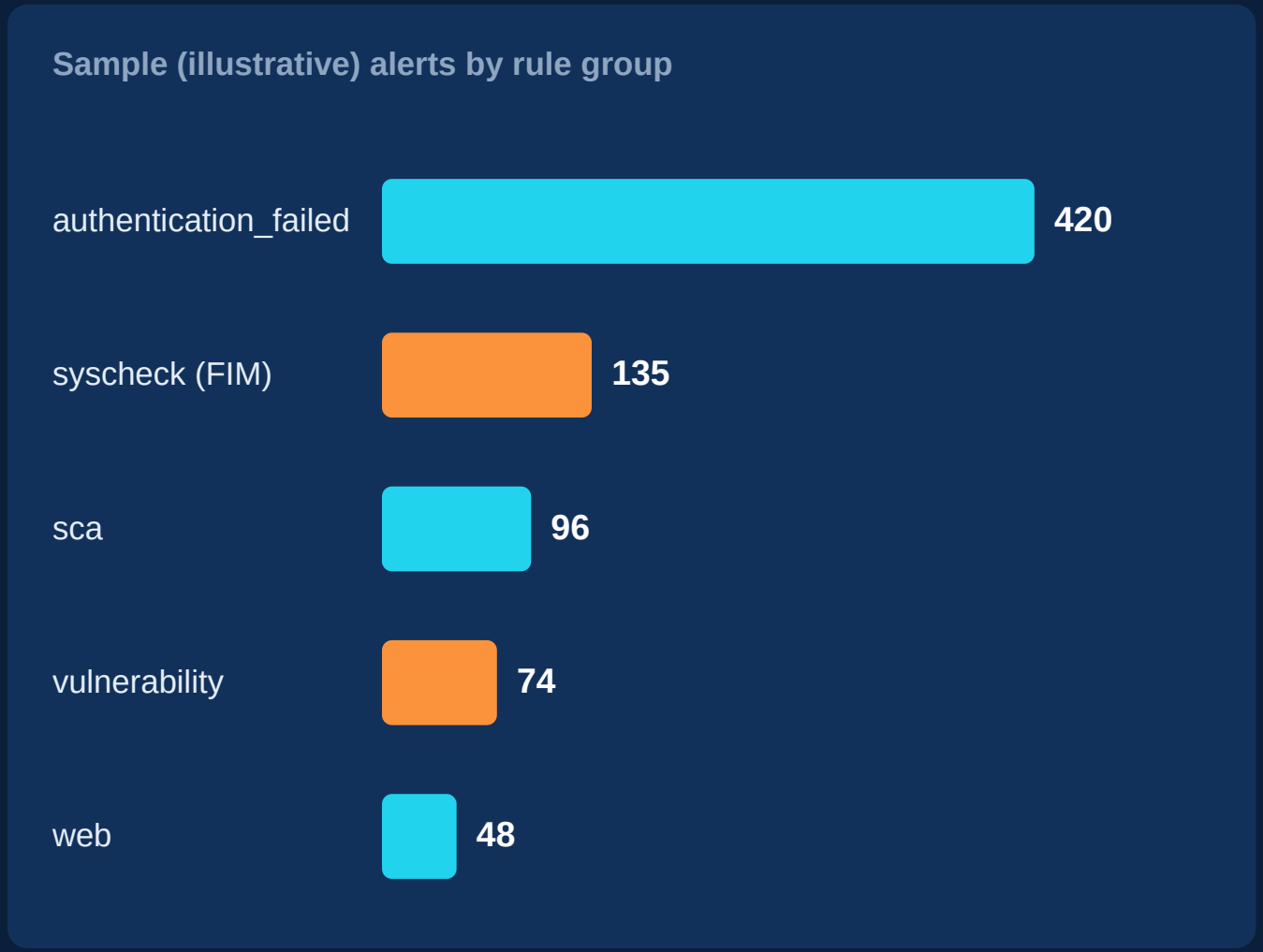
- **Safety rules**
  Whitelist admin and monitoring IPs, test on one agent first, and keep console access in case you block the wrong address.

Wazuh Use Case: SSH Brute Force



- FIM alert if sshd_config changes afterward
- Integrates with Slack, email and TheHive
- Replaces manual Fail2ban + auth.log checks

Wazuh: Reading the Dashboard



- **authentication_failed**
  the biggest group, usually scanners and brute force
- **syscheck**
  file changes, review the unexpected ones
- **sca + vulnerability**
  your hardening to-do list
- Filter by agent, rule group or MITRE technique

Wazuh: Troubleshooting and Checklist

Symptom	Likely cause	What to check
Agent never connects	Ports blocked or wrong manager address	`nc -zv <mgr> 1514, agent ossec.log`
Dashboard will not load	Indexer down or not enough RAM	`systemctl status wazuh-indexer, journalctl -u wazuh-dashboard`
Custom rule never fires	Wrong if_sid, XML typo, manager not restarted	`wazuh-logtest, ossec.log`
Manager will not start after edit	Invalid XML in config or rules	`/var/ossec/bin/wazuh-analysisd -t`
Alert flood	Noisy FIM paths or low-value rules	`Narrow <directories>, add <ignore>, raise rule level only if needed`
No vulnerability data	Feature disabled or inventory not synced yet	`Check vulnerability detection settings and agent inventory`

Before you call it done

- Change default passwords and store install files safely
- Use agent groups (web, db) for shared FIM and log settings
- Snapshot or back up the indexer data and /var/ossec/etc
- Allow 1514/1515 only from agent networks, keep 9200 and 55000 internal
- Whitelist admin IPs before enabling active response
- Upgrade the manager first, then agents (never newer than the manager)

Snort: Overview and Modes

- Network IDS/IPS created by Martin Roesch, now owned by Cisco
- Signature-based: compares packets to rules
- Rules come from Cisco Talos, the community, or your own
- Snort 3: multi-threaded, new rule syntax, Lua configuration

Three modes

Sniffer: shows packets on screen
Packet Logger: saves packets to disk
NIDS / NIPS: detection and prevention, the real use

Snort 2 vs Snort 3

	Snort 2 (legacy)	Snort 3 (current)
Configuration	snort.conf (custom syntax)	snort.lua (Lua scripting)
Threading	One packet thread per process	Multiple packet threads in one process
Rule syntax	Protocol only: tcp, udp, icmp, ip	Also service rules such as alert http, with sticky buffers
Reload	Restart to apply changes	Reload configuration while running
Alert output	Unified2 and fast / full	alert_fast, alert_full, alert_json, alert_csv, log_pcap
Design	Preprocessors	Modular inspectors and plugins
Status	Legacy, plan to migrate	Use for all new deployments

rule syntax difference

```
# Snort 2 style
alert tcp any any -> any 80 (content:"/admin"; ...)
# Snort 3 style, service rule and sticky buffer
alert http any any -> any any (http_uri; content:"/admin"; ...)
```

Most community rules work in both, but not all. Cisco ships a converter, snort2lua, to move an old snort.conf and its rules to Snort 3.

Snort: Packet Pipeline



- Every packet goes through the same stages
- Capture uses the DAQ library (afpacket, pcap) to read from an interface
- Inspectors rebuild streams and understand protocols before rules run
- The rule action decides what happens: alert, drop, or just log

Snort 3: How Detection Works in Detail

1 Packet source
DAQ: afpacket, pcap, file

→

2 Decode
Ethernet, IP, TCP/UDP headers

→

3 Inspectors
stream, port_scan, http, ssl, dns

→

4 Detection
fast pattern, then full rule

→

5 Logging
alert_fast, alert_json, log_pcap

Why Snort is fast: the fast-pattern funnel



Numbers are illustrative: the idea is to avoid testing every rule on every packet.

- **Inspectors rebuild what the server sees**
  stream_tcp reassembles the byte stream, http_inspect decodes URIs, so an attacker cannot hide a pattern by splitting packets or encoding characters.
- **Rules only inspect what they ask for**
  Sticky buffers such as http_uri; or http_header; point a content match at one field, which is faster and more accurate than searching the whole packet.
- **Actions decide the outcome**
  alert, log and pass work in IDS and IPS. drop, block and reject only act when Snort sits inline.

Memory tip: Snort = capture, decode, normalize, match, act. Every false positive or miss can be traced to one of those five stages.

Typical outputs: alert_fast (one line per alert), alert_json (for SIEM), log_pcap (save the packet for later study)

Snort: IDS vs IPS Deployment

IDS mode (passive)

- Connected to a switch mirror/SPAN port
- Watches a copy of the traffic
- Safe: cannot disrupt the network
- Alerts only, cannot block

IPS mode (inline)

- Traffic passes through Snort
- Can drop malicious packets
- Risk: false positives block real traffic
- Tune rules before enabling

> *Start in IDS mode. Review alerts for a few weeks, suppress noisy rules, then move only proven rules to drop.*

Snort 3 Install, Step 1: Dependencies and libdaq

tj@sensor: ~

```
$ sudo apt update
$ sudo apt install -y build-essential cmake git pkg-config autoconf libtool \
    libpcap-dev libpcre2-dev libdumbnet-dev libhwloc-dev libluajit-5.1-dev \
    libssl-dev zlib1g-dev libhyperscan-dev libunwind-dev liblzma-dev \
    uuid-dev libflatbuffers-dev flatbuffers-compiler bison flex
Reading package lists... Done
...
Setting up libhyperscan-dev:amd64 ...
Setting up cmake ...
```

tj@sensor: ~/snort_src (libdaq, the capture library)

```
$ mkdir -p ~/snort_src && cd ~/snort_src
$ git clone https://github.com/snort3/libdaq.git
Cloning into 'libdaq'...
$ cd libdaq && ./bootstrap && ./configure && make -j$(nproc)
...
$ sudo make install && sudo ldconfig
Libraries have been installed in:
   /usr/local/lib
# ldconfig refreshes the library cache so snort can find libdaq
```

tj@sensor: check the DAQ modules (sample)

```
$ ls /usr/local/lib/daq
daq_afpacket.so  daq_bpf.so  daq_dump.so  daq_nfq.so  daq_pcap.so  daq_savefile.so
```

Illustrative screenshot: sample output, values and versions will differ on your system.

1. **Build tools**
   build-essential, cmake and git compile Snort from source.
2. **Libraries**
   libpcap captures packets, libpcre2 does regex, libdumbnet and hwloc help with networking and CPU layout, LuaJIT runs snort.lua.
3. **Hyperscan**
   The fast pattern matcher. Optional, but strongly recommended for speed.
4. **libdaq first**
   Snort 3 will not build without the DAQ library. Always run sudo ldconfig after installing it.

> *Package names are for Ubuntu 22.04. On other distros use the equivalent -devel packages.*

Snort 3 Install, Step 2: Build Snort

tj@sensor: ~/snort_src

```
$ cd ~/snort_src && git clone https://github.com/snort3/snort3.git
Cloning into 'snort3'...
$ cd snort3 && ./configure_cmake.sh --prefix=/usr/local
---------------------------------------------------
snort version 3.x.x.x
Install options:
    prefix:    /usr/local
    includes:  /usr/local/include/snort
    plugins:   /usr/local/lib/snort
---------------------------------------------------
-- Configuring done
-- Build files have been written to: /home/tj/snort_src/snort3/build
$ cd build && make -j$(nproc)
[  4%] Building CXX object src/...
[ 37%] Building CXX object src/...
[ 81%] Linking CXX shared library ...
[100%] Built target snort
$ sudo make install
-- Installing: /usr/local/bin/snort
-- Installing: /usr/local/etc/snort/snort.lua
-- Installing: /usr/local/etc/snort/snort_defaults.lua
# If the build is killed (out of memory), retry with fewer jobs:  make -j2
```

Illustrative screenshot: sample output, values and versions will differ on your system. Output is shortened.

1. **Get the source**
   Official code is on GitHub (snort3/snort3). Release tarballs are on snort.org.
2. **configure_cmake.sh**
   A helper script that runs cmake for you. The prefix decides where files go, here /usr/local.
3. **make -j$(nproc)**
   Compiles using all CPU cores. Expect roughly 10 to 30 minutes, and lots of RAM.
4. **make install**
   Copies the snort binary and the default snort.lua into /usr/local.

> *You can also use distro packages or Docker images, but a source build gives the newest version and teaches you the layout.*

Snort 3 Install, Step 3: Verify

tj@sensor: ~

```
$ snort -V

   ,,_     -*> Snort++ <*-
  o"  )~   Version 3.x.x.x
   ''''    By Martin Roesch & The Snort Team
           Using DAQ version 3.x.x
           Using LuaJIT version 2.1.x
           Using libpcap version 1.10.x
           Using PCRE2 version 10.x
           Using Hyperscan version 5.x
```

tj@sensor: ~

```
$ snort -c /usr/local/etc/snort/snort.lua -T
Loading /usr/local/etc/snort/snort.lua:
        ...
Snort successfully validated the configuration (with 0 warnings).
o")~   Snort exiting
```

Where things live after a /usr/local install

`/usr/local/bin/snort`	the program
`/usr/local/etc/snort/snort.lua`	main configuration
`/usr/local/etc/rules/local.rules`	your own rules (create it)
`/var/log/snort/`	alert logs (create it)

Illustrative screenshot: sample output, values and versions will differ on your system.

1. **snort -V**
   Prints the version and the libraries it was built with. If Hyperscan is missing, rebuild with it.
2. **-T validates**
   Loads the whole configuration, then exits. Run it after every edit to snort.lua or a rule file.
3. **Create the folders**
   sudo mkdir -p /usr/local/etc/rules /var/log/snort, then add a local.rules file.

- **sudo cannot find snort?**
  sudo may ignore /usr/local/bin in its PATH. Use the full path or link it: sudo ln -s /usr/local/bin/snort /usr/bin/snort.

Snort: snort.lua Essentials

CONFIG

```
-- snort.lua (excerpt)
HOME_NET = '192.168.1.0/24'
EXTERNAL_NET = '!$HOME_NET'

ips =
{
    enable_builtin_rules = true,
    include = RULE_PATH .. '/local.rules',
    variables = default_variables
}

alert_fast = { file = true }

-- Validate from the shell:
-- snort -c /usr/local/etc/snort/snort.lua -T
```

- **HOME_NET**
  the network you protect, rules use it to know "inside" vs "outside"
- **ips**
  which rule files and built-in rules to load
- **alert_fast**
  one-line alerts written to the log folder
- Always validate with -T before running

Snort: Writing Rules

RULES

```
# local.rules
alert tcp any any -> $HOME_NET 22 (
  msg:"Possible SSH brute force";
  flags:S;
  detection_filter:track by_src, count 5, seconds 60;
  sid:1000001; rev:1;)

alert icmp any any -> $HOME_NET any (
  msg:"ICMP ping"; itype:8;
  sid:1000002; rev:1;)
```

- **Rule = header + options**
  Header: action proto src -> dst. Options in ( ).
- **sid**
  custom SIDs start at 1,000,000
- **rev**
  increase it each time you edit the rule
- **detection_filter**
  alert only after 5 SYNs from one source in 60 seconds

Anatomy of a Snort Rule

```
alert tcp any any -> $HOME_NET 22 ( ... )
```

Action — Protocol — Source IP, port — Direction — Destination IP, port — Options

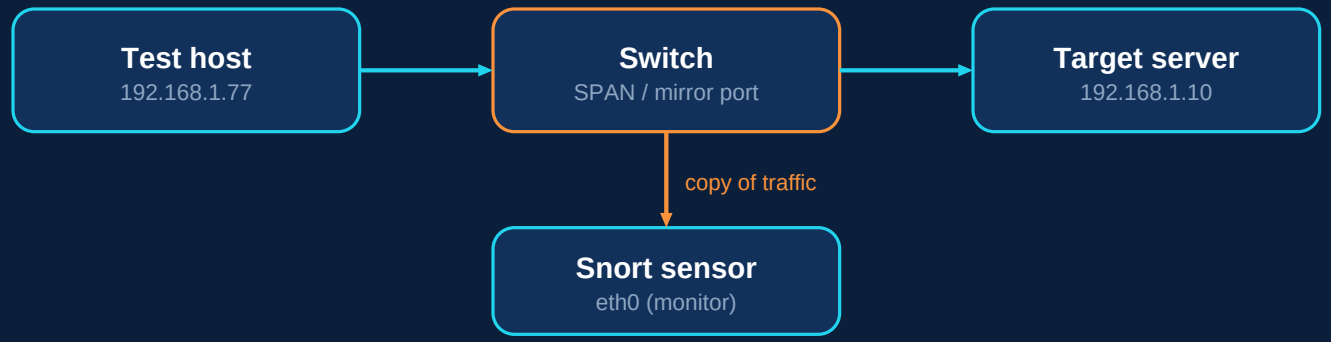
The header says WHERE to look. The options say WHAT to look for. Both must match for the rule to fire.

Option	Group	Meaning
`msg:"Possible SSH brute force";`	General	Text shown in the alert. Write something an analyst can act on.
`flags:S;`	Non-payload	Match only TCP packets with the SYN flag (connection attempts).
`detection_filter:track by_src, count 5, seconds 60;`	Post-detection	Alert only after 5 matches from one source within 60 seconds.
`sid:1000001; rev:1;`	General	Unique rule ID (custom rules start at 1,000,000) and revision number.
`content:"/etc/passwd"; nocase;`	Payload	Search the data for a string. nocase ignores upper or lower case.
`http_uri;`	Payload buffer	Sticky buffer: the next content match applies only to the URI.

a payload rule (web attack example)

```
alert http any any -> $HOME_NET any (
    msg:"Suspicious UNION SELECT in URI"; http_uri; content:"union select", nocase;
    classtype:web-application-attack; sid:1000003; rev:1;)
```

Testing Your Rules in a Lab



sensor: run Snort in IDS mode

```
$ sudo snort -c /usr/local/etc/snort/snort.lua -R /usr/local/etc/rules/local.rules \
    -i eth0 -A alert_fast -k none
Commencing packet processing
++ [0] eth0
11/07-10:42:15.318244 [**] [1:1000002:1] "ICMP ping" [**] [Priority: 0] {ICMP}
    192.168.1.77 -> 192.168.1.10
11/07-10:43:02.114870 [**] [1:1000001:1] "Possible SSH brute force" [**]
    [Priority: 0] {TCP} 192.168.1.77:51422 -> 192.168.1.10:22
```

1

2

test host: generate harmless test traffic (your own lab only)

```
$ ping -c 3 192.168.1.10
$ for i in $(seq 1 6); do nc -zw1 192.168.1.10 22; done
# 6 SYNs to port 22 within 60 s is above the detection_filter count of 5
```

3

1. **Run in IDS mode first**
   Use a mirror port so a mistake cannot block traffic. -k none skips checksum checks that fail on mirrored traffic.
2. **Read the alert line**
   [1:1000002:1] is gid:sid:rev. Then the message, protocol and the two endpoints.
3. **Trigger it on purpose**
   Every rule you write needs a test that makes it fire, and one that proves it stays quiet on normal traffic.

> *No mirror port? Test offline: snort -c snort.lua -R local.rules -r test.pcap -A alert_fast*

Illustrative screenshot: sample output, values and versions will differ on your system.

Snort: Commands

COMMANDS

```
# Validate config, then run (IDS)
snort -V
snort -c /usr/local/etc/snort/snort.lua -T
sudo snort -c snort.lua -R local.rules \
  -i eth0 -A alert_fast -k none

# Read a pcap
snort -c snort.lua -R local.rules \
  -r capture.pcap -A alert_fast

# Inline IPS
sudo snort -c snort.lua -R local.rules \
  --daq afpacket -i eth0:eth1 -Q
```

- **-T**
  test configuration and exit
- **-R**
  load a rules file
- **-r**
  read a saved capture, ideal for testing rules safely
- **-Q**
  inline mode, two interfaces bridged
- **-k none**
  skip checksum checks on a mirror port

Tuning and Rule Updates

snort.lua (tuning excerpt)

```
-- 1) Silence one noisy rule for one trusted host
suppress =
{
    { gid = 1, sid = 1000002, track = 'by_src', ip = '192.168.1.5' },
}
-- 2) Limit volume: at most 1 alert per minute per source
event_filter =
{
    { gid = 1, sid = 1000001, type = 'limit',
      track = 'by_src', count = 1, seconds = 60 },
}
-- 3) Load an extra rule file
ips =
{
    include = RULE_PATH .. '/snort3-community.rules',
}
```

Rule set	Cost	Notes
Community	Free	Small, good for learning
Registered	Free account	Larger, released with a delay
Subscriber	Paid	Newest rules from Talos
Emerging Threats Open	Free	Third-party community rules
local.rules	Yours	Rules for your own network

Terms and delays change, always check snort.org. PulledPork3 can download and combine rule sets for you.

Tuning workflow



Tuning is a routine, not a one-time job. Review the top 10 alerting SIDs every week, record why each was kept, suppressed or limited, and only move rules that have proven reliable into drop mode.

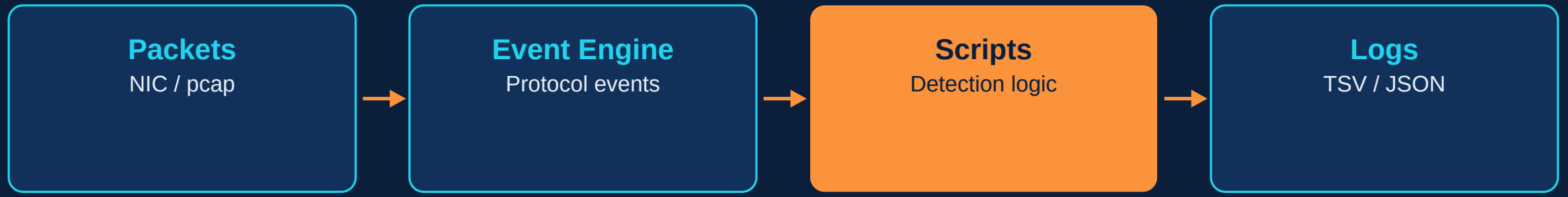
Snort: Troubleshooting and Best Practices

Symptom or error	Likely cause	Fix
error while loading shared libraries: libdaq.so	Library cache not refreshed	`sudo ldconfig`
snort: command not found (with sudo)	/usr/local/bin not in sudo PATH	`Use the full path or add a symlink`
Config test fails on a rule file	Typo, missing ; or wrong path	`snort -c snort.lua -T and read the line number`
No alerts, but traffic exists	Wrong interface or mirror port not seeing traffic	`ip -s link, tcpdump -i eth0 -c 5, check SPAN`
Missing or odd packets on a busy link	NIC offload merges packets	`ethtool -K eth0 gro off lro off`
Packet drops, high CPU	Too many rules or too few threads	`Trim unused rules, try --max-packet-threads N`
All traffic stops in inline mode	Wrong bridge or interface pair	`Return to IDS mode, check -i eth0:eth1 and the cabling`

Best practices

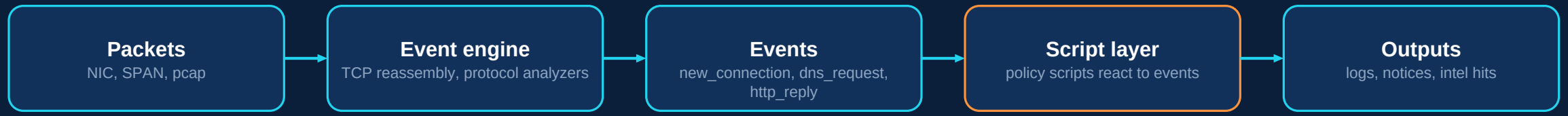
- Start in IDS mode and review alerts for a few weeks
- Set HOME_NET correctly, wrong values cause silent misses
- Run -T after every change and keep configs in git
- Send alerts to Wazuh or a SIEM, not only to a local file

Zeek: How It Works



- Formerly Bro (Vern Paxson, 1990s), renamed Zeek in 2018
- A Network Security Monitor, not mainly signature-based
- Passive: watches traffic and writes detailed logs, has its own scripting language
- Used in Security Onion, research networks and enterprise SOCs
- **Like a court stenographer**
  records who connected to whom, which DNS names were asked and which certificates were used

Zeek: Architecture in Detail



Standalone (lab)

NIC / SPAN port

one zeek process
sniffs, analyzes and writes logs

Cluster (production, many Gbps)

Worker 1
sniffs a share

Worker 2
sniffs a share

Worker 3
sniffs a share

Proxy
shares state

Manager
coordinates

Logger
writes the logs

zeekctl
starts, stops and deploys all nodes

Workers do the heavy work.
Scale by adding more workers.

Log framework

every analyzer writes structured logs: conn, dns, http, ssl

Notice framework

turns suspicious events into notice.log entries

Intel framework

matches traffic against lists of bad IPs, domains, hashes

Files framework

extracts file info and hashes from any protocol

Zeek: The Logs

Log	Contents	Example use
`conn.log`	Every connection: IPs, ports, duration, bytes	Exfiltration, beaconing
`dns.log`	DNS queries and answers	Malware domains, DNS tunneling
`http.log`	Requests, user-agent, URI	Suspicious web activity
`ssl.log`	TLS version, certificates, SNI	Self-signed or expired certs
`files.log`	Files seen, with hashes	Malware hash lookup
`notice.log`	Events flagged as suspicious	Alert triage

Zeek Install, Step 1: Add the Repo and Install

tj@sensor: ~

```
$ lsb_release -ds
Ubuntu 22.04.5 LTS
# 1) add the Zeek package repository
$ echo 'deb http://download.opensuse.org/repositories/security:/zeek/xUbuntu_22.04/ /' \
    | sudo tee /etc/apt/sources.list.d/security:zeek.list
# 2) trust its signing key
$ curl -fsSL https://download.opensuse.org/repositories/security:zeek/xUbuntu_22.04/Release.key \
    | gpg --dearmor | sudo tee /etc/apt/trusted.gpg.d/security_zeek.gpg > /dev/null
# 3) install (zeek-lts = long-term support, zeek = newest features)
$ sudo apt update && sudo apt install -y zeek-lts
...
Setting up zeek-lts ...
# 4) Zeek lives in /opt/zeek, add it to your PATH
$ echo 'export PATH=/opt/zeek/bin:$PATH' >> ~/.bashrc && source ~/.bashrc
$ zeek --version
zeek version 7.x.x
$ ls /opt/zeek
bin  etc  include  lib  logs  share  spool
```

Illustrative screenshot: sample output, values and versions will differ on your system.

1. **Repository**
   The OpenSUSE build service hosts official Zeek packages. Change xUbuntu_22.04 to match your release.
2. **Signing key**
   apt refuses unsigned repos. The key lets it verify the packages.
3. **zeek vs zeek-lts**
   LTS gets fixes for longer and is the safer choice for a long-running sensor.
4. **Add to PATH**
   Everything installs under /opt/zeek, so commands like zeek and zeekctl are not found until you add it.

> *Other ways: the zeek/zeek Docker image, building from source, or Security Onion, which ships Zeek preconfigured.*

Zeek Install, Step 2: Configure the Sensor

tj@sensor: find the capture interface

```
$ ip -br a
lo      UNKNOWN  127.0.0.1/8 ::1/128
eth0    UP       192.168.1.20/24
eth1    UP       <- connected to the SPAN / TAP port
```

1

/opt/zeek/etc/node.cfg and networks.cfg (after editing)

```
$ grep -v '^#' /opt/zeek/etc/node.cfg
[zeek]
type=standalone
host=localhost
interface=eth1
$ grep -v '^#' /opt/zeek/etc/networks.cfg
10.0.0.0/8          Private IP space
192.168.0.0/16      Private IP space
```

2

/opt/zeek/share/zeek/site/local.zeek (lines added at the end)

```
# JSON logs, easy for Wazuh and jq
@load policy/tuning/json-logs
# add file hashes (MD5/SHA1) to files.log
@load policy/frameworks/files/hash-all-files
# mirror ports often show bad checksums because of NIC offload
redef ignore_checksums = T;
```

3

1. **Choose the right NIC**
   Zeek must listen on the mirror or TAP port, not the management port that has your IP.
2. **node.cfg and networks.cfg**
   node.cfg sets the node type and interface. networks.cfg lists your local ranges so logs can tell inside from outside.
3. **local.zeek**
   Your site policy. Each @load enables a script. Keep changes small and test them one by one.

> *With ignore_checksums on, Zeek accepts packets with checksums that are wrong only because of offloading.*

Illustrative screenshot: sample output, values and versions will differ on your system.

Zeek: Deployment

ZEEKCTL

```
# /opt/zeek/etc/node.cfg
[zeek]
type=standalone
host=localhost
interface=eth0

# /opt/zeek/etc/networks.cfg
10.0.0.0/8        Private
192.168.0.0/16    Private

# local.zeek: JSON logs for SIEM
@load policy/tuning/json-logs

sudo /opt/zeek/bin/zeekctl deploy
sudo /opt/zeek/bin/zeekctl status
```

- **standalone**
  one machine does everything, fine for labs
- **interface**
  should be a SPAN port or network TAP
- **Logs**
  in /opt/zeek/logs/current/, rotated into dated folders
- Paths depend on how Zeek was installed

First Run: Deploy and Check the Logs

tj@sensor: ~

```
$ sudo /opt/zeek/bin/zeekctl deploy
checking configurations ...
installing ...
creating policy directories ...
installing site policies ...
generating standalone-layout.zeek ...
generating local-networks.zeek ...
stopping ...
starting ...
starting zeek ...
$ sudo /opt/zeek/bin/zeekctl status
Name         Type       Host          Status    Pid    Started
zeek         standalone localhost     running   12891  07 Nov 10:55:12
```

1

2

tj@sensor: ~

```
$ ls /opt/zeek/logs/current
capture_loss.log  conn.log  dns.log  files.log  http.log  packet_filter.log
reporter.log  ssl.log  stats.log  stderr.log  stdout.log
$ curl -s http://example.com > /dev/null; sleep 5; tail -n 1 /opt/zeek/logs/current/http.log
```

3

crontab -e (root)

```
# keep logs rotating and restart crashed nodes
*/5 * * * * /opt/zeek/bin/zeekctl cron
```

4

1. **zeekctl deploy**
   Checks, installs and (re)starts. Run it after every config change.
2. **status running**
   If the status is crashed, run zeekctl diag to see why.
3. **logs/current**
   Fresh logs appear here once traffic is seen. Each hour they rotate into dated folders and are compressed.
4. **zeekctl cron**
   Run every 5 minutes. It rotates logs and restarts nodes that crashed.

Illustrative screenshot: sample output, values and versions will differ on your system.

Zeek: Analyzing Logs

ZEEK-CUT

```
cd /opt/zeek/logs/current

# Biggest uploads
cat conn.log | zeek-cut id.orig_h id.resp_h orig_bytes \
  | sort -k3 -nr | head

# Top talkers
cat conn.log | zeek-cut id.orig_h \
  | sort | uniq -c | sort -nr | head

# Most asked DNS names
cat dns.log | zeek-cut query | sort | uniq -c | sort -nr | head

# Replay a pcap
zeek -r capture.pcap
```

With JSON logs enabled, use jq instead of zeek-cut. Replaying a pcap writes fresh logs into the current folder.

Reading conn.log

/opt/zeek/logs/current

```
$ cat conn.log | zeek-cut ts id.orig_h id.orig_p id.resp_h id.resp_p proto service duration orig_bytes resp_bytes conn_state | head -6
ts            orig_h        o_port resp_h          r_port proto service duration o_byt  r_byt  state
1794013322.41 192.168.1.25  51544  142.250.4.101   443    tcp   ssl     12.84    1873   9412   SF
1794013325.02 192.168.1.25  53221  192.168.1.1     53     udp   dns     0.03     38     54     SF
1794013330.77 192.168.1.40  44100  192.168.1.10    22     tcp   -       -        -      -      S0      <- SYN, no answer: scan?
1794013331.10 192.168.1.40  44102  192.168.1.10    23     tcp   -       0.00     0      0      REJ     <- port closed (rejected)
1794013401.55 192.168.1.31  49820  198.51.100.7    8443   tcp   ssl     0.51     220    180    SF      <- same pair ...
1794013461.57 192.168.1.31  49837  198.51.100.7    8443   tcp   ssl     0.50     220    180    SF      <- 60 s later: beaconing?
```

conn_state	Meaning	Why it matters
S0	SYN seen, no reply	Scans, dead hosts, blocked traffic
SF	Normal start and finish	Healthy connection
REJ	Attempt rejected	Probe of a closed port
RSTO / RSTR	Reset by originator / responder	Abrupt end, or a firewall at work
OTH	No SYN seen, started midstream	Capture began mid-connection

history letter	Meaning
S / H	SYN / SYN-ACK (handshake)
A	ACK
D	Data packet
F / R	FIN (close) / RST (reset)
UPPER vs lower	UPPER = from originator, lower = from responder

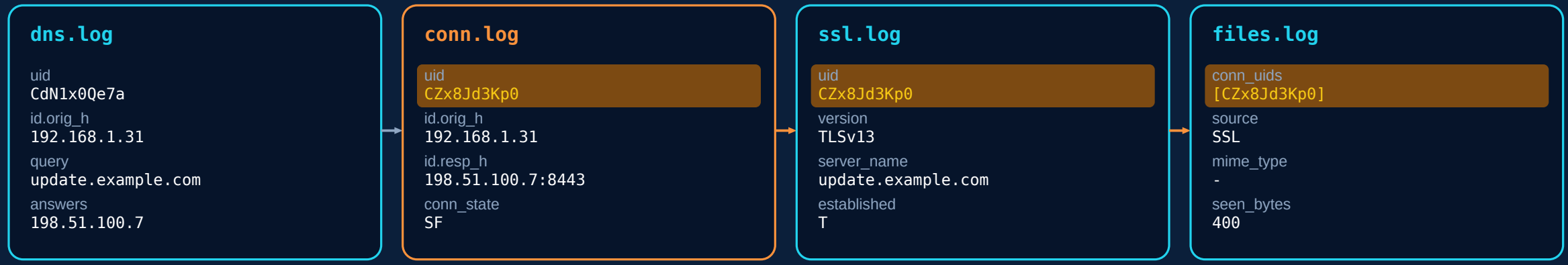
Tip

zeek-cut output is tab-separated and has no header. A dash (-) means the field is unset. Add -d to zeek-cut to turn the epoch time into a readable date.

Example: history = ShADadFf

A normal full connection: handshake (S, h, A), data both ways (D, d), then a clean close (F, f). A history of just S means the target never answered.

Following One Connection Across Logs



Step 1: DNS gives the IP. Step 2: the connection to that IP has a unique uid. Steps 3 and 4: the same uid links the other logs.

tj@sensor: /opt/zeek/logs/current (TSV logs)

```
# who asked for update.example.com, and what IP did DNS return?
$ cat dns.log | zeek-cut id.orig_h query answers | grep update.example.com
192.168.1.31   update.example.com   198.51.100.7
# find that connection, then pull every log line with the same uid
$ cat conn.log | zeek-cut uid id.orig_h id.resp_h id.resp_p | grep 198.51.100.7
CZx8Jd3Kp0   192.168.1.31   198.51.100.7   8443
$ grep -l CZx8Jd3Kp0 *.log        # which logs mention it?
conn.log  files.log  ssl.log
```

- **JSON logs**
  Use jq instead: jq 'select(.uid=="CZx8Jd3Kp0")' *.log

- **Link to other tools**
  The Community ID (a Zeek package) lets you match a Zeek connection to alerts from tools that also compute it.

Zeek: Scripts and Threat Hunting

SCRIPT

```
# long-dns.zeek
event dns_request(c: connection, msg: dns_msg,
                  query: string, qtype: count, qclass: count)
{
  if ( |query| > 50 )
    print fmt("Long DNS from %s: %s",
              c$id$orig_h, query);
}

# Run it on a pcap
zeek -r capture.pcap long-dns.zeek
```

- **Beaconing**
  regular check-ins in conn.log suggest malware C2
- **DNS tunneling**
  very long domain names
- **Retrospective search**
  when a new IOC appears, search old logs for it

Threat Hunting Recipes

Beaconing

```
cat conn.log | zeek-cut id.orig_h id.resp_h id.resp_p | sort | uniq -c | sort -nr | head
```

RED FLAG

One internal host talking to one rare external IP and port hundreds of times, with the same small byte counts at fixed intervals.

DNS tunneling

```
cat dns.log | zeek-cut query | awk '{print length($0), $0}' | sort -nr | head
```

RED FLAG

Very long, random-looking subdomains under one domain, or a flood of TXT queries to a single name.

Large uploads (exfiltration)

```
cat conn.log | zeek-cut id.orig_h id.resp_h orig_bytes | sort -k3 -nr | head
```

RED FLAG

A workstation sending far more data out than it receives, to an unknown address, especially at odd hours.

Odd clients

```
cat http.log | zeek-cut user_agent | sort | uniq -c | sort -n | head
```

RED FLAG

Rare user agents such as curl or python-requests on office PCs, or a user agent seen only once.

retrospective search: a new bad IP was published today, was it seen last week?

```
$ zgrep -h '198.51.100.7' /opt/zeek/logs/2026-11-0*/conn.*.log.gz | wc -l
```

Zeek: Troubleshooting and Best Practices

Symptom	Likely cause	What to check
Logs empty, only stats.log	Wrong interface or no traffic on the SPAN port	`ip -s link, tcpdump -i eth1 -c 5, node.cfg`
zeekctl deploy fails	Script error in local.zeek	`zeekctl check, then zeekctl diag`
Missing or strange connections	NIC offload, bad checksums	`ignore_checksums, disable offload on the NIC`
Packet loss in capture_loss.log	One process cannot keep up	`Move to a cluster with several workers`
Disk fills up	Logs kept forever	`zeekctl cron, LogExpireInterval in zeekctl.cfg`
zeek-cut prints nothing	Logs are JSON, not TSV	`Use jq instead of zeek-cut`

Best practices

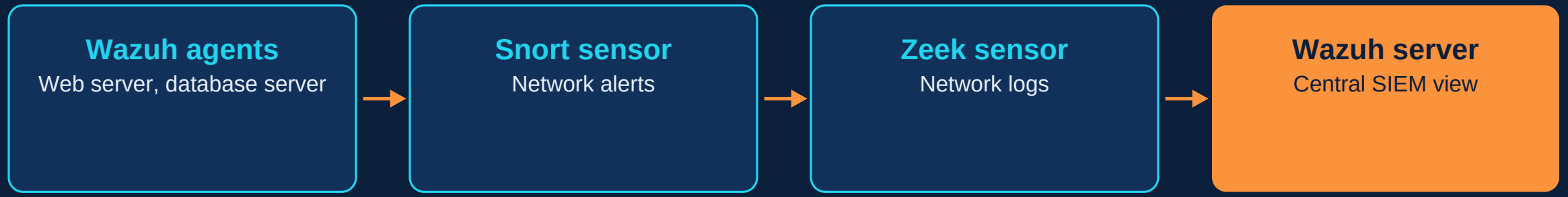
- Put the sensor on a SPAN or TAP that sees both directions
- Watch capture_loss.log, missing packets mean missing evidence
- Set networks.cfg so local and external hosts are labeled correctly
- Forward JSON logs to Wazuh or a SIEM and keep the raw logs for hunting

Comparison: Wazuh vs Snort vs Zeek

	Wazuh	Snort	Zeek
Scope	Host	Network	Network
Method	Logs, FIM, rules	Signatures	Metadata + scripts
Can block?	Yes (Active Response)	Yes (IPS mode)	No (passive)
Output	Alerts + dashboard	Alerts	Rich logs
Best at	Endpoint, compliance	Known attacks, blocking	Forensics, hunting

> *Snort = guard at the gate. Zeek = CCTV with recording. Wazuh = guard inside each room.*

Layered Setup: Better Together



- Network + host correlation: scan seen by Snort + connection in Zeek + failed SSH logins in Wazuh = one clear attack story
- Put a Wazuh agent on each sensor so Snort alerts and Zeek logs reach the same dashboard
- One place to search, one place to alert

Integration: Snort and Zeek Logs into Wazuh

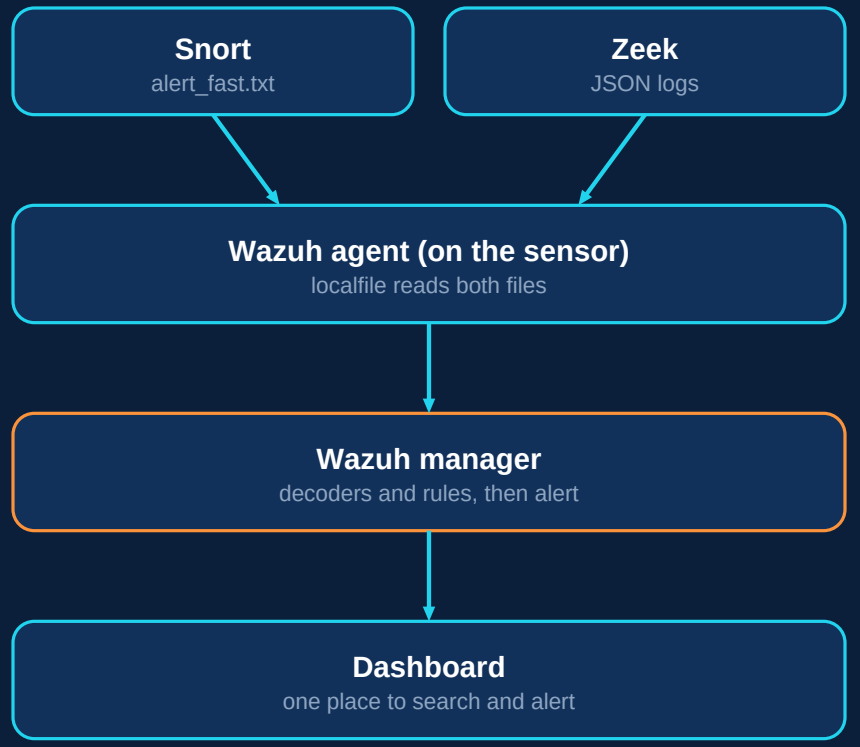
sensor: /var/ossec/etc/ossec.conf (Wazuh agent on the sensor)

```
<!-- Snort alerts (fast format) -->
<localfile>
  <log_format>snort-fast</log_format>
  <location>/var/log/snort/alert_fast.txt</location>
</localfile>

<!-- Zeek notices, JSON format -->
<localfile>
  <log_format>json</log_format>
  <location>/opt/zeek/logs/current/notice.log</location>
</localfile>

$ sudo systemctl restart wazuh-agent
```

- **Snort**
  Wazuh already knows the Snort alert formats, so alerts show up with rule groups like ids.
- **Zeek**
  JSON logs are parsed automatically. Write custom rules (100000 and up) for the fields you care about, such as a notice type.



Start small: notice.log first. conn.log and dns.log are very large and can flood the indexer.

Paths and log names depend on how you installed each tool.

Alert Triage Workflow



- **Zeek conn.log**
  shows everything the suspect host talked to
- **Wazuh**
  shows what changed on the host: logins, files, processes
- Every false positive is a tuning task, not just noise

RECAP

Summary and Recommendations

- **Wazuh**
  host guard: logs, FIM, vulnerabilities, active response
- **Snort**
  network guard: signatures, optional blocking
- **Zeek**
  network recorder: rich logs, hunting
- Start with Wazuh (easiest to deploy), then add Snort or Zeek
- Always tune rules to reduce false positives

RESOURCES

References and Further Reading

Tool	Where to learn more	What you will find
Wazuh	documentation.wazuh.com	Installation guide, rule and decoder syntax, FIM and active response
Wazuh	packages.wazuh.com	Installer script and agent packages
Snort	docs.snort.org	Snort 3 manual, rule writing, configuration reference
Snort	snort.org/downloads	Source code, rule sets and community rules
Zeek	docs.zeek.org	Installation, log reference, scripting language guide
Zeek	zeek.org	Releases, packages and community resources
General	attack.mitre.org	MITRE ATT&CK, used by Wazuh to label detections
General	cisecurity.org/cis-benchmarks	CIS benchmarks, the basis for Wazuh SCA checks

> *All installation commands and versions in this deck were written for a lab on Ubuntu 22.04. Always check the official documentation for the latest steps before installing on a real system.*

Q & A

Detect. Analyze. Respond.